

2do Parcial 1era fecha 2025

Ejercicio 2

Parte A

Clasifique las siguientes estructuras de datos de acuerdo a lo visto en la catedra

```
1  class Moto:
2      def __init__(self):
3          self._patente = None
4          self._anio = None
5          self._modelo = None
6          self._kms = []
7
8      @property
9      def anio(self):
10         return self._anio
11
12     @anio.setter
13     def anio(self, valor):
14         self._anio = valor
15
16     def get_kms(self, km):
17         return self._kms[km]
```

Aca podemos ver un producto cartesiano en la clase Moto ya que agrupa un conjunto de n-tuplas de elementos heterogeneos bajo una misma unidad y podemos ver una correspondencia finita en Moto._kms ya que hay un conjunto de indices mapeados hacia un conjunto de resultados

```
1  type
2      PVertice = ^Vertice;
3      PAdyacente = ^Adyacente;
4
5      // Lista de adyacencia (punteros a otros vértices)
6      Adyacente = record
7          destino: PVertice;
8          siguiente: PAdyacente;
9      end;
10
11     // Nodo del grafo
12     Vertice = record
```

```

13     id: Integer;
14     adyacentes: PAdyacente;
15     siguiente: PVertice;
16 end;

```

Aca podemos ver dos constructores recursivos, uno en Vertice, ya que el mismo contiene un puntero a otro Vertice lo cual hace que pueda crecer infinitamente. El otro caso esta en Adyacente, el cual contiene un puntero a Adyacente por lo que puede crecer infinitamente. Tambien hay un producto cartesiano en Adyacente y Vertice, ya que agrupan un conjunto de n-tuplas de elementos heterogeneos

Parte B

a) **No existen lenguajes compilados que sean debilmente tipados, basicamente por poder chequear estaticamente el tipo**

Falso . Existen lenguajes como C que son debilmente tipados y a la vez compilados. Una cosa es en que momento se chequean los tipos (compilacion) y la otra que tan estrictos son con ese chequeo

b) **En Java, implementar una clase, asegura de por si que ya posee encapsulamiento y ocultamiento**

Falso . En Java se puede crear una clase y dejar todos sus campos publicos, no romperia el encapsulamiento pero si romperia el ocultamiento. El ocultamiento de la informacion se logra ocultando los datos y detalles de la implementacion

Ejercicio 3

```

1  public class ParcialExcepciones {
2      public static void main(String[] args) {
3          try{
4              for (int i = 0; i < 5; i++) {
5                  if(i==1){
6                      System.out.println(Integer.toString(i));
7                      relanzador(i);
8                  }
9                  else{
10                     if(i==2 || i==3){
11                         switch(i){
12                             case 2:
13                                 System.out.println(Integer.toString(i));
14                                 relanzador(i);
15                                 break;

```

```

16             case 3:
17                 System.out.println(Integer.toString(i));
18                 relanzador(i);
19                 break;
20             }
21         }
22         else{
23             System.out.println(Integer.toString(i));
24             relanzador(i);
25         }
26     }
27 }
28 }catch(ThirdException | FourthException e){
29     System.out.println(e.getMessage());
30 }
31 }
32
33 static void relanzador(int i) throws ThirdException, FourthException {
34     try {
35         try {
36             switch(i){
37                 case 1:
38                     throw new FirstException(Integer.toString(i));
39                     break;
40                 case 2:
41                     throw new SecondException(Integer.toString(i));
42                     break;
43                 case 3:
44                     throw new ThirdException(Integer.toString(i));
45                     break;
46                 default:
47                     throw new FourthException(Integer.toString(i));
48                     break;
49             }
50         }catch (SecondException e) {
51             ThirdException e1=new ThirdException(Integer.toString(i));
52             throw e1;
53         }
54     }catch (FirstException e) {
55         FourthException e1=new FourthException(Integer.toString(i));
56         throw e1;
57     }
58 }
59 }

```

a)

i) Se imprime en pantalla "0", luego "0" y termina Correcta

ii) Se imprime en pantalla "0", "0", "1", "1", "2", "2", "3", "3", "4", "4" y luego termina

iii) Se imprime en pantalla "0", "0", "1", "1", "2", "2", "3", "3" y luego termina

iv) Ninguna de las anteriores

Lo que pasa aca es que el for esta adentro del catch, asi que cuando encuentra el 0, se ejecuta el catch y termina la ejecucion, imprimira la b si el try estuviera adentro del for

b)

Lo que pasa aca es que el primer numero lanzado es el 1, entra al primer if y llama al relanzador. Una vez ahi lanza una FirstException, la cual no puede ser resuelto por el catch local, asi que avanza y va al siguiente catch, el cual si puede atraparla y relanza una nueva FourthException con el mensaje 1. La FourthException se atrapa en el main y se vuelve a imprimir su mensaje, por lo que se termina imprimiendo "1" y "1"

a. La unión y la unión discriminada son igualmente seguras. Falso

b. Java tiene un equivalente a la sentencia YIELD de python. Falso

c. El switch en Java no requiere la cláusula default de manera obligatoria. Verdadero

d. En PL/1 si se genera una excepción, se ejecuta el manejador correspondiente y luego el control se pasa inmediatamente a la rutina padre de aquella donde se generó la excepción.

Falso

e. La sentencia finally de python en el manejo de excepciones se ejecuta siempre y cuando no se haya levantado una nueva excepción dentro de un except. Falso

Justificaciones

a. La union discrimanda da la capacidad de preguntar con que dato estoy tratando

b. Java no tiene un equivalente, solo tiene la capacidad de hacer return, el cual devuelve un valor y mata el registro de activacion

c. No se requiere, si un valor no coincide con niguna rama no se ejecuta nada y sigue la ejecucion normal

d. Si se genera una excepcion y se ejecuta el manejador correspondiente, vuelve el control a la linea siguiente donde se produjo el error

e. Esa es la sentencia `else` , generalmente finally lo que hace es ejecutar un bloque de codigo en cualquier caso, tanto como si se atrapo o no una excepcion